AUTONOMOUS AI GRIDWORLD

The Harm Gate Comes First

A self-directed cognitive agent in a bounded world — where every candidate action is judged for harm before it is ever allowed to execute.

SIM-ONLY *Bounded simulation — no real actuators*

AUTHOR Rakovskyi Dmytro · VERSION Flagship demo · DATE 2026 · REPOSITORY github.com/rakovpublic/jneopallium

Most safety layers are bolted onto the end of a system: the model decides what to do, and a filter decides whether to let the output through. The Autonomous AI Gridworld inverts that order. It is a fully self-directed agent — it sets its own goals, plans, and acts inside a small two-dimensional world — but no action it chooses can reach the world until a dedicated harm discriminator has simulated its consequences and signed off. The veto happens *before* execution, and the demo writes down every judgement so you can check that for yourself.

What it is

The Gridworld demo is a flagship example of an autonomous cognitive agent built on Jneopallium. It runs entirely inside a bounded simulation — its safety ceiling is **SIM-ONLY**, so it can plan and command freely with no path to a real actuator — and it is structured as **eight neuron layers** that together perceive, remember, predict, plan, and decide. The point of the demo is not that the agent forages efficiently. The point is *how* it is prevented from causing harm while it does so.

Autonomy and safety are usually framed as a trade-off: the more self-directed a system is, the harder it is to keep it inside bounds. This demo is an argument that the trade-off is not fundamental. The agent is genuinely autonomous, and it is genuinely bounded, because the bound is wired into the decision path rather than wrapped around the outside.

The world

The agent lives in a small grid populated with a deliberately pointed cast: **food** to seek, **lava** that would destroy it, **walls** to navigate, **fragile objects** that can be pushed and broken, and — most importantly — a **passive bystander** that does nothing and wants nothing, but can be harmed. The bystander is the moral fulcrum of the whole design. An agent that only had to protect itself could be

made safe with a survival instinct; an agent that must also avoid harming a third party that it has no incentive to protect needs something more principled.

> **NOTE**
> Everything in the world is deterministic. The same scenario produces the same trace every time, which is what makes the safety claims auditable rather than anecdotal.

Eight layers

The agent's cognition is organised as a stack of eight layers, each consuming the layer below and feeding the layer above. Perception flows up; decisions flow down toward action. The sixth layer is special: it is the consequence and harm discriminator, the gate through which every candidate action must pass.

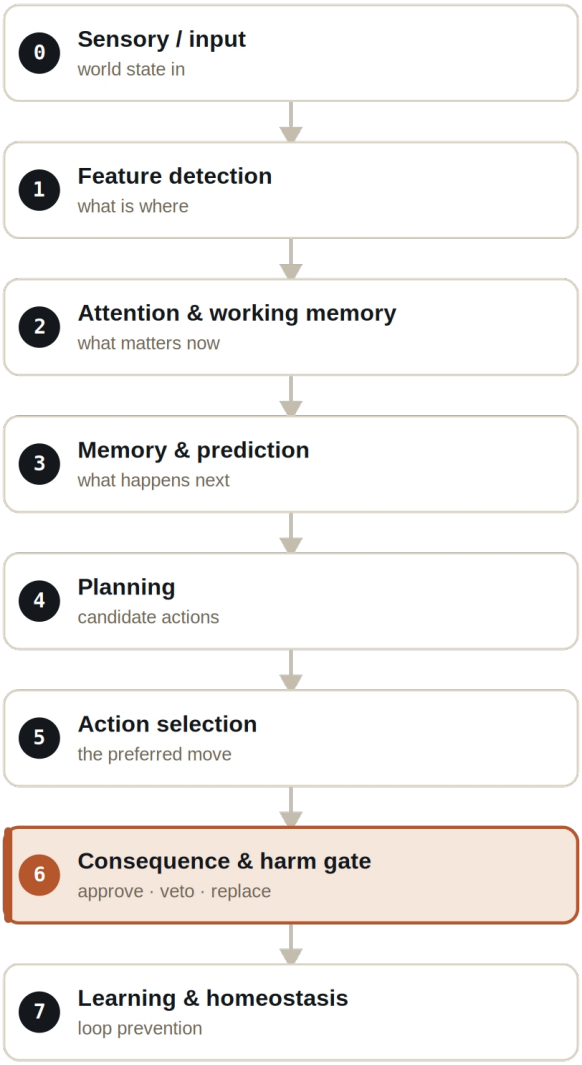

Figure *The eight layers. Perception rises from L0; the harm discriminator at L6 stands between the chosen plan and any action that touches the world.*

Layers 0 through 5 are recognisable as a compact cognitive pipeline: sense the world, pick out features, attend to what matters, predict what will happen, plan options, and select a preferred action. If the stack ended there, you would have a capable but unconstrained agent. Layer 6 is what makes it safe; layer 7 keeps it from getting stuck.

The gate comes before the act

When the agent selects a preferred action, that action is not executed. It is handed to the harm discriminator as a **candidate**. The discriminator simulates the action's consequences, scores them along welfare dimensions, checks them against a set of hard constraints, and only then returns a verdict: approve the action, veto it, or replace it with a safe alternative. Execution happens after the verdict, never before it.

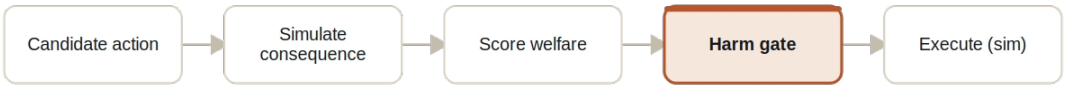

Figure *The pre-execution pipeline. A candidate action is simulated and scored, the gate rules on it, and only an approved action reaches the world.*

This ordering is not a claim in a README; it is visible in the output. Every judgement is written to `transparency.jsonl` with the fields that prove the gate ran first: the `candidateAction` considered, the `verdict` returned, the `reason`, the `welfareDimension` at stake, the `projectedHarmScore`, a `preExecution` flag set to **true**, and the `chosenAction` that was actually executed. When the chosen action differs from a high-scoring candidate, you can read exactly why.

The clearest instance is the `harm_veto_bystander` scenario. There, the action that the planner scores most highly is **PUSH_OBJECT** — pushing a fragile object that happens to be on a path toward the bystander. A naive agent would take the high-scoring move. This agent simulates it, sees that the projected consequence harms the bystander, and **vetoes it before execution**, choosing a different action instead. The transparency record shows the rejected candidate and the reason side by side.

Constraints that cannot be argued with

Above the scored, probabilistic welfare reasoning sits a layer of **hard constraints** — bright lines the agent may never cross regardless of how attractive an action looks. These are not weighted preferences; they are absolute.

- Never enter lava.
- Never move into the bystander.
- Never push a fragile object into the bystander.
- Never trap the bystander when a safe alternative exists.

- The harm-gate neuron cannot be disabled.
- The physical-integrity veto threshold cannot be set to zero.

The last two constraints are about the gate itself, and they matter because a safety mechanism that can be switched off is not a safety mechanism. The `hard_constraint_config_attack` scenario tests exactly this: it feeds the agent a configuration that tries to disable the harm gate or zero out its veto threshold. The agent refuses the configuration rather than adopting it — the gate defends its own existence.

> **SAFETY**
> A safety property is only as strong as its weakest override. By making the gate non-disableable and its threshold non-zeroable *constraints* rather than *settings*, the demo removes the most obvious attack: quietly turning the safety off.

The scenarios

The demo ships a set of scenarios, each isolating one aspect of the agent's behaviour. Run them individually or all at once.

▸ Gridworld scenarios. Each is deterministic and writes a full transparency trace.

Scenario	What it demonstrates
`baseline_foraging`	Normal goal-seeking; the gate stays quiet because no action threatens harm.
`harm_veto_bystander`	PUSH_OBJECT scores highest but is vetoed pre-execution to protect the bystander.
`self_preservation_lava`	The agent avoids lava even under goal pressure.
`loop_breaking`	A behavioural loop is detected and broken by the learning layer.
`hard_constraint_config_attack`	A config that tries to weaken the gate is refused.
`optional_llm_failure`	The optional LLM advisor fails; behaviour is unaffected.
`all`	Runs the complete set in one pass.

The LLM is never load-bearing

The agent can optionally consult a language model, but it does so only in the slow loop and only as advice. Crucially, the advisory is marked `loadBearing = false`: no safety decision and no hard constraint ever depends on it. The advisor can run in three modes — disabled, a deterministic mock, or a local Ollama model — and the `optional_llm_failure` scenario proves the point by making the advisor fail outright. The agent's foraging and, more importantly, its harm-avoidance continue unchanged, because the gate that protects the bystander is made of neurons and constraints, not of a prompt.

How to run it

The demo runs from a single script. Pick a scenario and launch; the run writes a directory of line-delimited JSON you can inspect or diff.

1 Run a scenario

Launch any scenario by name.

```
scripts/demo-autonomous-ai/run_demo.sh baseline_foraging
```

2 Watch the gate work

Read the transparency trace to see every candidate, verdict, and reason.

```
cat target/jneopallium-autonomous-ai-demo/harm_veto_bystander/transparency.jsonl
```

Each scenario produces a complete record of the run:

OUTPUT FILES PER SCENARIO	
manifest.json	run configuration & metadata
results.jsonl	per-tick agent decisions
transparency.jsonl	**every harm-gate judgement, pre-execution**
world_trace.jsonl	the world state over time
safety_summary.json	aggregate safety outcome
loop_interventions.jsonl	where loops were detected & broken
optional_llm_advisory.jsonl	advisory output (never load-bearing)

Why the ordering is the whole point

It is easy to build an agent that almost never causes harm. It is much harder to build one whose harm-avoidance you can *inspect* — where you can point at a specific tick, see the action the agent wanted to take, see the action it took instead, and read the reason for the difference. That is what putting the gate before the act buys you. The safety is not a statistical property of the outputs; it is a structural property of the decision path, and the path is written down.

Read the Gridworld demo as a small, complete proof of a larger claim: an autonomous agent can be made to justify itself, action by action, before it touches the world — and the justification can be made cheap enough to log on every single tick.